GAZİ UNIVERSITY

FACULTY OF ENGINEERING
DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 10: Filters-WHERE, AND, OR, LIMIT, FETCH

FILTERS

WHERE, AND Operator, OR Operator, LIMIT, FETCH

PostgreSQL WHERE clause

The `SELECT` statement returns all rows from one or more columns in a table. To retrieve rows that satisfy a specified condition, you use a `WHERE` clause.

The syntax of the PostgreSQL `WHERE` clause is as follows:

```
SELECT
  select_list
FROM
  table_name
WHERE
  condition
ORDER BY
  sort_expression;
```

PostgreSQL WHERE clause

In this syntax, you place the `WHERE` clause right after the `FROM` clause of the `SELECT` statement.

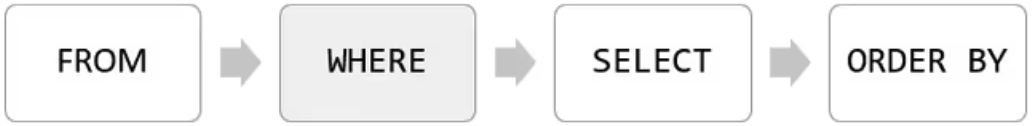
The `WHERE` clause uses the `condition` to filter the rows returned from the `SELECT` clause.

The `condition` is a boolean expression that evaluates to true, false, or unknown.

The query returns only rows that satisfy the `condition` in the `WHERE` clause. In other words, the query will include only rows that cause the `condition` to evaluate to true in the result set.

PostgreSQL WHERE clause

PostgreSQL evaluates the `WHERE` clause after the `FROM` clause but before the `SELECT` and `ORDER BY` clause:



If you use column aliases in the `SELECT` clause, you cannot use them in the `WHERE` clause.

Besides the `SELECT` statement, you can use the `WHERE` clause in the `UPDATE` and `DELETE` statement to specify rows to update and delete.

PostgreSQL WHERE clause

- To form the condition in the WHERE clause, you use comparison and logical operators:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<> or !=	Not equal

Operator	Description
AND	Logical operator AND
OR	Logical operator OR
IN	Return true if a value matches any value in a list
BETWEEN	Return true if a value is between a range of values
LIKE	Return true if a value matches a pattern
IS NULL	Return true if a value is NULL
NOT	Negate the result of other operators

PostgreSQL WHERE clause-Examples

customer
* customer_id
store_id
first_name
last_name
email
address_id
activebool
create_date
last_update
active

- The following statement uses the WHERE clause to find customers whose first name is Jamie:

```
SELECT last_name, first_name FROM customerWHERE first_name = 'Jamie';
```

- AND, OR logical operator to find customers whose first name and last names are Jamie and Rice:

```
SELECT last_name, first_name FROM customer WHERE first_name = 'Jamie' AND last_name = 'Rice';
```

```
SELECT first_name, last_name FROM customer WHERE last_name = 'Rodriguez' OR first_name = 'Adam';
```

- IN operator to find the customers with first names in the list Ann, Anne, and Annie:

```
SELECT first_name, last_name FROM customer WHERE first_name IN ('Ann', 'Anne', 'Annie');
```

- LIKE operator in the WHERE clause to find customers whose first names start with the word Ann:

```
SELECT first_name, last_name FROM customer WHERE first_name LIKE 'Ann%';
```

PostgreSQL WHERE clause-Examples

customer
* customer_id
store_id
first_name
last_name
email
address_id
activebool
create_date
last_update
active

- The following example finds customers whose first names start with the letter A and contains 3 to 5 characters by using the BETWEEN operator.
- The BETWEEN operator returns true if a value is in a range of values.

```
SELECT first_name, LENGTH(first_name) name_length FROM customer
WHERE first_name LIKE 'A%' AND LENGTH(first_name) BETWEEN 3 AND 5
ORDER BY name_length;
```

- Finding customers whose first names start with Bra and last names are not Motley:

```
SELECT first_name, last_name FROM customer WHERE first_name LIKE 'Bra%' AND last_name <> 'Motley';
```

➢ Note that you can use the != operator and <> operator interchangeably because they are equivalent.

PostgreSQL WHERE clause-Practice

- Create customer table and insert the data then try to answer questions in the following slide:

```
CREATE TABLE customer (
    customer_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    city VARCHAR(50),
    country VARCHAR(50),
    age INT,
    email VARCHAR(100),
    active BOOLEAN
);
```

```
INSERT INTO customer (first_name, last_name, city, country, age, email, active)
VALUES
('Jamie', 'Rice', 'New York', 'USA', 32, 'jamie.rice@example.com', TRUE),
('Jamie', 'Waugh', 'Los Angeles', 'USA', 28, 'jamie.waugh@example.com', TRUE),
('Adam', 'Gooch', 'Chicago', 'USA', 35, 'adam.gooch@example.com', FALSE),
('Laura', 'Rodriguez', 'Madrid', 'Spain', 29, 'laura.rodriguez@example.com', TRUE),
('Ann', 'Evans', 'London', 'UK', 41, 'ann.evans@example.com', TRUE),
('Anne', 'Powell', 'Manchester', 'UK', 37, 'anne.powell@example.com', FALSE),
('Annie', 'Russell', 'Dublin', 'Ireland', 26, 'annie.russell@example.com', TRUE),
('Anna', 'Hill', 'Berlin', 'Germany', 24, 'anna.hill@example.com', TRUE),
('Annette', 'Olson', 'Oslo', 'Norway', 33, 'annette.olson@example.com', FALSE),
('Brandy', 'Graves', 'Toronto', 'Canada', 31, 'brandy.graves@example.com', TRUE),
('Brandon', 'Huey', 'Vancouver', 'Canada', 27, 'brandon.huey@example.com', TRUE),
('Brad', 'Mccurdy', 'Sydney', 'Australia', 45, 'brad.mccurdy@example.com', FALSE);
```

PostgreSQL WHERE clause-Practice

- Write a query to find all customers whose first_name is 'Jamie'.
- Write a query to find all customers who live in 'USA' and are active.
- Write a query to find all customers whose first_name starts with 'Ann'.

PostgreSQL AND Operator

In PostgreSQL, a boolean value can have one of three values: `true`, `false`, and `null`.

PostgreSQL uses `true`, `'t'`, `'true'`, `'y'`, `'yes'`, `'1'` to represent `true` and `false`, `'f'`, `'false'`, `'n'`, `'no'`, and `'0'` to represent `false`.

A boolean expression is an expression that evaluates to a boolean value. For example, the expression `1=1` is a boolean expression that evaluates to `true`:

```
SELECT 1 = 1 AS result;
```

Output:

```
result
--------
 t
(1 row)
```

The letter t in the output indicates the value of true.

PostgreSQL AND Operator

The `AND` operator is a logical operator that combines two boolean expressions.

Here's the basic syntax of the `AND` operator:

```
expression1 AND expression2
```

In this syntax, `expression1` and `expression2` are boolean expressions that evaluate `true`, `false`, or `null`.

The `AND` operator returns `true` only if both expressions are `true`. It returns `false` one of the expressions is `false`. Otherwise, it returns `null`.

The following table shows the results of the `AND` operator when combining `true`, `false`, and `null`. Note that the order of the expressions doesn't matter, for examp both `true AND null` and `null AND true` will evaluate to `null`.

PostgreSQL AND Operator

The following table shows the results of the `AND` operator when combining `true`, `false`, and `null`. Note that the order of the expressions doesn't matter, for example both `true AND null` and `null AND true` will evaluate to `null`.

expression1	expression2	expression1 AND expression2
True	True	True
True	False	False
True	Null	Null
False	False	False
False	Null	False
Null	Null	Null

PostgreSQL AND Operator-Examples

The following example uses the AND operator to combine values, which returns according to the table:

```
SELECT true AND true AS result;
```

Output:

```
 result
--------
 t
(1 row)
```

```
SELECT true AND false AS result;
```

Output:

```
 result
--------
 f
(1 row)
```

```
SELECT true AND null AS result;
```

Output:

```
 result
--------
 null
(1 row)
```

```
SELECT false AND false AS result;
```

Output:

```
 result
--------
 f
(1 row)
```

PostgreSQL AND Operator-Examples

film
* film_id
title
description
release_year
language_id
rental_duration
rental_rate
length
replacement_cost
rating
last_update
special_features
fulltext

The following example uses the AND operator in the WHERE clause to find the films that have **a length greater than 180** and **a rental rate less than 1**:

```
SELECT title, length, rental_rate FROM film WHERE length > 180 AND rental_rate < 1;
```

Output:

```
title         | length | rental_rate
--------------------+--------+-------------
 Catch Amistad      |    183 |        0.99
 Haunting Pianist   |    181 |        0.99
 Intrigue Worst     |    181 |        0.99
 Love Suicides      |    181 |        0.99
 Runaway Tenenbaums |    181 |        0.99
 Smoochy Control    |    184 |        0.99
 Sorority Queen     |    184 |        0.99
 Theory Mermaid     |    184 |        0.99
 Wild Apollo        |    181 |        0.99
 Young Language     |    183 |        0.99
(10 rows)
```

Answer This Question now:

Using the film table, write a SQL query to return the distinct rental_rate values for films where length > 120, replacement_cost > 20, and rental_rate < 3, and sort the results by rental_rate in ascending order.

PostgreSQL OR Operator

In PostgreSQL, a boolean value can have one of three values: `true`, `false`, and `null`.

PostgreSQL uses `true`, `'t'`, `'true'`, `'y'`, `'yes'`, `'1'` to represent `true` and `false`, `'f'`, `'false'`, `'n'`, `'no'`, and `'0'` to represent `false`.

A boolean expression is an expression that evaluates to a boolean value. For example, the expression `1<>1` is a boolean expression that evaluates to `false`:

```
SELECT 1 <> 1 AS result;
```

Output:

```
result
--------
 f
(1 row)
```

The letter f in the output indicates false.

PostgreSQL OR Operator

The `OR` operator is a logical operator that combines multiple boolean expressions. Here's the basic syntax of the `OR` operator:

```
expression1 OR expression2
```

In this syntax, `expression1` and `expression2` are boolean expressions that evaluate to `true`, `false`, or `null`.

The `OR` operator returns `true` only if any of the expressions is `true`. It returns `false` if both expressions are false. Otherwise, it returns null.

PostgreSQL OR Operator

The following table shows the results of the `OR` operator when combining `true`, `false`, and `null`. Note that the order of the expressions doesn't matter, for example both `false OR null` and `null OR false` will evaluate to `null`.

expression1	expression2	expression1 OR expression2
True	True	True
True	False	True
True	Null	True
False	False	False
False	Null	Null
Null	Null	Null

PostgreSQL OR Operator-Examples

The following example uses the AND operator to combine values, which returns according to the table:

```
SELECT true OR true AS result;
```

Output:

```
result
--------
 t
(1 row)
```

```
SELECT true OR false AS result;
```

Output:

```
result
--------
 t
(1 row)
```

```
SELECT true OR null AS result;
```

Output:

```
result
--------
 t
(1 row)
```

```
SELECT false OR false AS result;
```

Output:

```
result
--------
 f
(1 row)
```

PostgreSQL OR Operator-Examples

film
* film_id
title
description
release_year
language_id
rental_duration
rental_rate
length
replacement_cost
rating
last_update
special_features
fulltext

The following example uses the OR operator in the WHERE clause to find the films that have a rental rate of 0.99 or 2.99:

```
SELECT title, rental_rate FROM film WHERE rental_rate = 0.99 OR rental_rate = 2.99;
```

Output:

```
title              | rental_rate
-------------------------------+-------------
 Academy Dinosaur              |        0.99
 Adaptation Holes              |        2.99
 Affair Prejudice              |        2.99
 African Egg                   |        2.99
...
```

Answer This Question now:

Using the film table, write a SQL query to return the title, length, and rental_rate for films where length > 180 orrental_rate < 1, and sort the results by title in ascending order.

PostgreSQL LIMIT clause

PostgreSQL `LIMIT` is an optional clause of the `SELECT` statement that constrains the number of rows returned by the query.

Here's the basic syntax of the `LIMIT` clause:

```
SELECT
  select_list
FROM
  table_name
ORDER BY
  sort_expression
LIMIT
  row_count;
```

The statement returns `row_count` rows generated by the query. If the `row_count` is zero, the query returns an empty set. If the `row_count` is `NULL`, the query returns the same result set as it does not have the `LIMIT` clause.

PostgreSQL LIMIT clause

- If you want to skip a number of rows before returning the row_count rows, you can use OFFSET clause placed after the LIMIT clause.
- The statement first skips row_to_skip rows before returning row_count rows generated by the query.
- If the row_to_skip is zero, the statement will work as if it doesn't have the OFFSET clause.

```
SELECT
  select_list
FROM
  table_name
ORDER BY
  sort_expression
LIMIT
  row_count
OFFSET
  row_to_skip;
```

- It's important to note that PostgreSQL evaluates the OFFSET clause before the LIMIT clause.
- PostgreSQL stores rows in a table in an unspecified order, therefore, when you use the LIMIT clause, you should always use the ORDER BY clause to control the row order.
- If you don't use the ORDER BY clause, you may get a result set with the rows in an unspecified order.

PostgreSQL LIMIT clause

film
* film_id
title
description
release_year
language_id
rental_duration
rental_rate
length
replacement_cost
rating
last_update
special_features
fulltext

- To get the first five films sorted by film_id:

```
SELECT film_id, title, release_year FROM film ORDER BY film_id LIMIT 5;
```

- To retrieve 4 films starting from the fourth one ordered by film_id, you can use both LIMIT and OFFSET clauses as follows:

```
SELECT film_id, title, release_year FROM film ORDER BY film_id LIMIT 4 OFFSET 3;
```

- Using LIMIT to get top/bottom N rows

```
SELECT film_id, title, rental_rate FROM film ORDER BY rental_rate DESC LIMIT 10;
```

How it works

- First, sort all the films by rental rates from high to low using the ORDER BY rental_rate DESC clause.
- Second, take only 10 rows from the top using the LIMIT 10 clause.

PostgreSQL FETCH clause

- To skip a certain number of rows and retrieve a specific number of rows, you often use the LIMIT clause in the SELECT statement.
- The LIMIT clause is widely used by many Relational Database Management Systems such as MySQL, H2, and HSQLDB. However, the LIMIT clause is not a SQL standard.
- To conform with the SQL standard, PostgreSQL supports the FETCH clause to skip a certain number of rows and then fetch a specific number of rows.
- Note that the FETCH clause was introduced as a part of the SQL standard in SQL:2008.

PostgreSQL FETCH clause

The following illustrates the syntax of the PostgreSQL FETCH clause:

```
OFFSET row_to_skip { ROW | ROWS }
FETCH { FIRST | NEXT } [ row_count ] { ROW | ROWS } ONLY
```

- OFFSET specifies how many rows to skip, starting from 0 by default.
- If it skips more rows than exist, no rows are returned.
- FETCH specifies how many rows to return, defaulting to 1. ROW/ROWS and FIRST/NEXT mean the same thing.
- Since row order is not guaranteed, use FETCH with ORDER BY for consistent results.
- In PostgreSQL, OFFSET and FETCH can appear in any order.

PostgreSQL FETCH clause

film
* film_id
title
description
release_year
language_id
rental_duration
rental_rate
length
replacement_cost
rating
last_update
special_features
fulltext

- The following query uses the FETCH clause to select the first film sorted by titles in ascending order:

```
SELECT film_id, title FROM film ORDER BY title FETCH FIRST ROW ONLY;
```

- The following query uses the FETCH clause to select the first film sorted by titles in ascending order:

```
SELECT film_id, title FROM film ORDER BY title FETCH FIRST 5 ROW ONLY;
```

- The following statement returns the next five films after the first five films sorted by titles:

```
SELECT film_id, title FROM film ORDER BY title OFFSET 5 ROWS FETCH FIRST 5 ROW ONLY;
```

EXAMPLES

- Given following examples table and values try to write SQL commands for the given conditions using the filter.

  CREATE TABLE film (film_id INT PRIMARY KEY, title TEXT, description TEXT, release_year INT, language_id INT, rental_duration INT, rental_rate DECIMAL(4,2), length INT, replacement_cost DECIMAL(5,2), rating TEXT, last_update TIMESTAMP, special_features TEXT, fulltext TEXT);

  INSERT INTO film (film_id, title, rental_rate, length, rating, replacement_cost) VALUES (1, 'Academy Dinosaur', 0.99, 86, 'PG', 20.99), (2, 'Ace Goldfinger', 4.99, 48, 'G', 12.99), (3, 'Affair Prejudice', 2.99, 117, 'G', 26.99), (4, 'African Egg', 2.99, 130, 'G', 22.99), (5, 'Agent Truman', 2.99, 169, 'PG', 17.99), (6, 'Airplane Sierra', 4.99, 62, 'PG-13', 28.99), (7, 'Alabama Devil', 2.99, 114, 'PG-13', 21.99), (8, 'Aladdin Calendar', 4.99, 63, 'NC-17', 24.99);

QUESTIONS

- Find all unique rating categories, but only for films that are longer than 100 minutes AND have a rental_rate that is not 0.99. Your result should not show any duplicate ratings.
- Identify the title and length of movies that meet these specific criteria: They must have a rating of either 'G' or 'NC-17', AND they must either cost more than 25.00 to replace OR be shorter than 60 minutes. Use parentheses to ensure your logic is correct.
- Management needs to see the top 3 'budget-friendly' films. Write a query to find the title, rental_rate, and replacement_cost for movies that have a rental_rate under 3.00. Sort these by the highest replacement_cost first, then by title alphabetically. Use the FETCH clause to return only the first 3 results.

THE END

CHECK OUT COURSE WEBSITE: HTTPS://GAZI-END-MUH.GITHUB.IO

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM